accenture

Containerization with Docker

Building, Shipping, and Running Containers

Chapter Overview

01 Containers Fundamentals

02 Docker Fundamentals

03 Images and Containers

04 Dockerfiles and Layer Caching

05 Multi-Stage Builds

06 Docker Compose

07 Security and Production

Copyright © 2026 Accenture. All rights reserved.

Containers Fundamentals

Section 1



What is a Container?

Container basics

- A container is an isolated process on a host OS
- Uses host kernel features (not a full OS)
- Packages application code, dependencies, and runtime
- Does NOT include a kernel — shares the host kernel

Containers Are NOT Docker

- Containers are a Linux-native concept — Docker is one implementation
- Other tools (Podman, containerd, CRI-O) also run containers



Copyright © 2026 Accenture. All rights reserved.

Containers and Linux

Namespaces

- Process, network, and filesystem isolation — each container sees its own world

cgroups (Control Groups)

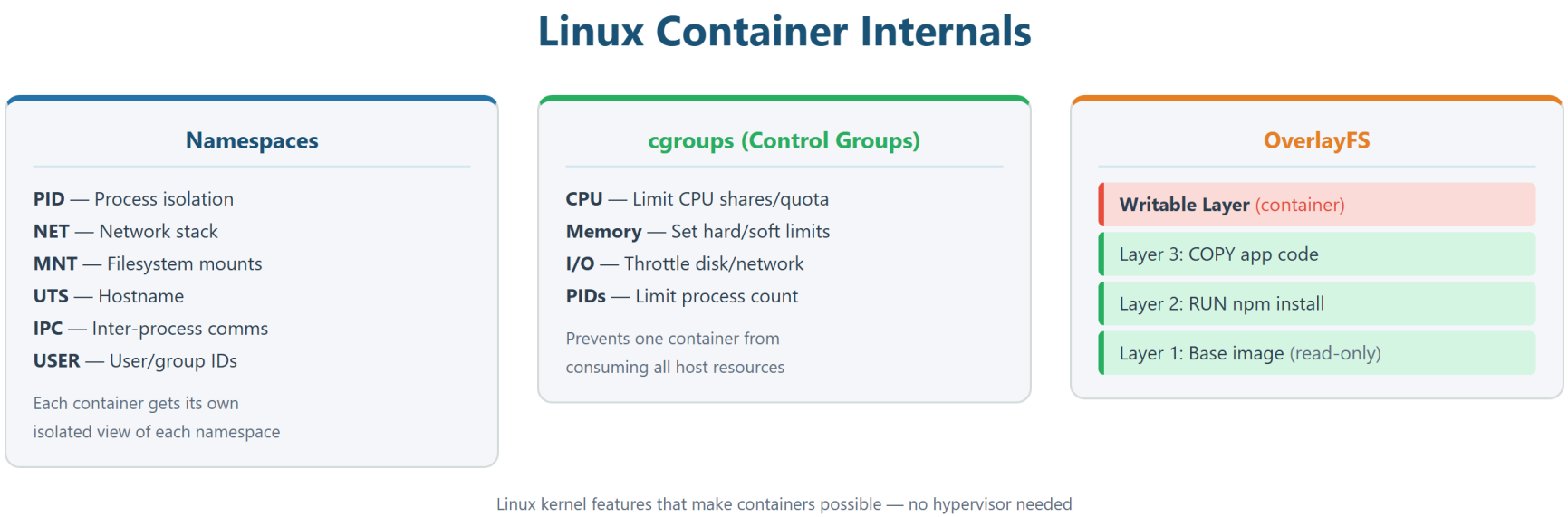
- CPU, memory, and I/O limits — prevent one container from starving others

OverlayFS (Layered Filesystem)

- Read-only image layers stacked with a thin writable layer on top

Copyright © 2026 Accenture. All rights reserved.

Linux Container Internals



Copyright © 2026 Accenture. All rights reserved.

Container Runtimes

Key Runtimes

- containerd — industry-standard runtime used by Docker and Kubernetes
- CRI-O — lightweight runtime purpose-built for Kubernetes
- runc — low-level OCI runtime that actually creates container processes
- Podman — daemonless alternative to Docker CLI (rootless by default)

What Docker Actually Is

- Docker = developer tooling + UX
- containerd/runc = the actual runtime



Copyright © 2026 Accenture. All rights reserved.

OCI Standard

Open Container Initiative (OCI)

- Industry standard defining how containers are built and run

Image Specification

- Defines the format for container images — layers, manifests, config
- Any OCI-compliant tool can build and run these images

Runtime Specification

- Defines how to run a container — filesystem, namespaces, cgroups config

Portability

- Build with Docker, run with Podman, orchestrate with Kubernetes — OCI enables this

Copyright © 2026 Accenture. All rights reserved.

Containers vs Virtual Machines

Containers

- Share host kernel — no guest OS overhead
- Start in milliseconds — just a process fork
- Megabytes in size — only app + dependencies

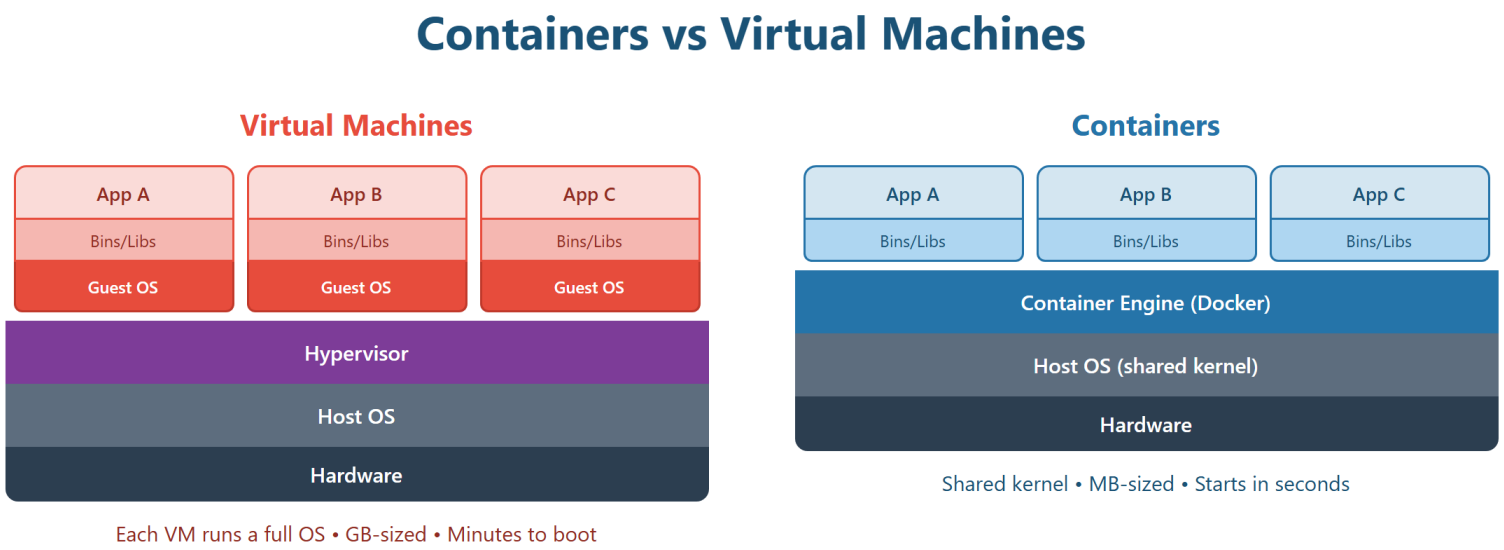
Virtual Machines

- Include a full guest OS and kernel — significant resource overhead
- Take seconds to minutes to boot
- Gigabytes in size — full OS image



Copyright © 2026 Accenture. All rights reserved.

Containers vs Virtual Machines



Copyright © 2026 Accenture. All rights reserved.

Docker Fundamentals

Section 2



Why Containers?

The Problem

- 'It works on my machine' — different environments cause deployment bugs
- Installing dependencies manually is error-prone and slow to reproduce

What Containers Solve

- Package your app with all its dependencies into a single portable unit
- Consistent behavior across environments (given compatible OS/kernel)

How Containers Work

- Containers are isolated processes — not lightweight VMs
- OverlayFS provides a layered filesystem — read-only image layers + writable container layer



Copyright © 2026 Accenture. All rights reserved.

Docker Ecosystem

Docker Engine

- The runtime that builds images and runs containers on your machine
- Runs as a daemon — uses containerd + runc internally
- Flow: CLI → Docker daemon → containerd → runc

Docker Hub

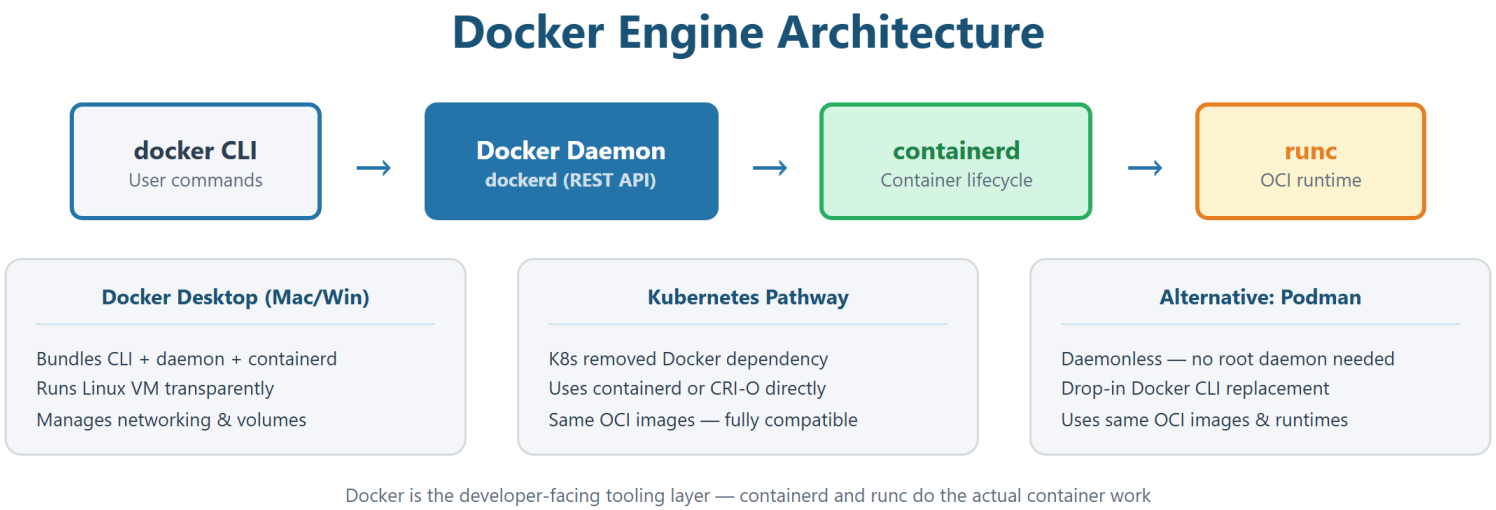
- Public registry with thousands of pre-built images (nginx, postgres, node)
- Official images are maintained and regularly scanned for vulnerabilities

Private Registries

- AWS ECR, GitHub Container Registry, self-hosted — for proprietary images

Copyright © 2026 Accenture. All rights reserved.

Docker Engine Architecture



Copyright © 2026 Accenture. All rights reserved.

Install and Verify Docker

```
# Verify Docker installation
docker --version
# Docker version 27.1.1, build 6312585

# Run your first container
docker run hello-world

# Run an interactive Ubuntu container
docker run -it ubuntu bash
root@a1b2c3d4:/# cat /etc/os-release
root@a1b2c3d4:/# exit

# Check Docker system info
docker info | head -20
```



Copyright © 2026 Accenture. All rights reserved.

Images and Containers

Section 3



Images vs Containers

Images

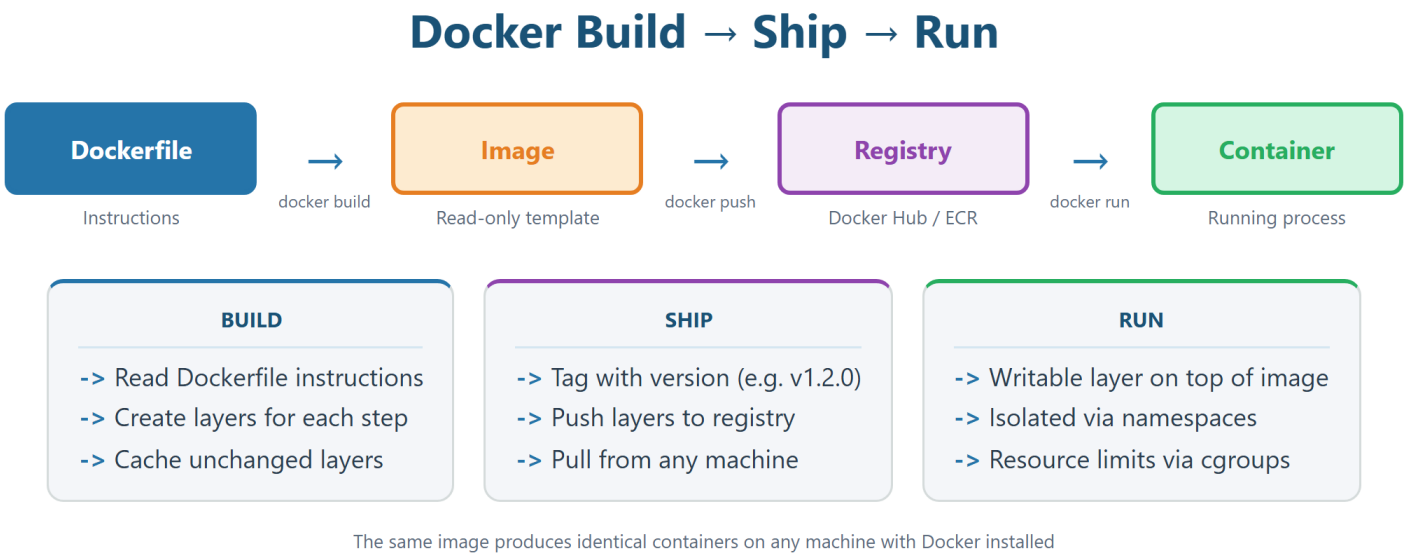
- Read-only templates — blueprint for creating containers
- Built from Dockerfiles, stored in registries (Docker Hub, ECR)
- Composed of stacked layers — each Dockerfile instruction adds a layer

Containers

- Running instances of images — isolated processes with their own filesystem
- Each container gets a thin writable layer on top of the image layers
- Writable layer persists across restarts but is lost when container is removed unless external storage is used

Copyright © 2026 Accenture. All rights reserved.

Docker Build-Ship-Run Lifecycle



Copyright © 2026 Accenture. All rights reserved.

Essential Container Commands

```
# Run a container in background with port mapping
docker run -d -p 3000:3000 --name api my-app

# List running containers (add -a for stopped ones)
docker ps
docker ps -a

# View container logs (follow mode)
docker logs -f api

# Execute a command inside a running container
docker exec -it api sh

# Stop, then remove the container
docker stop api && docker rm api
```



Copyright © 2026 Accenture. All rights reserved.

Container Lifecycle and Troubleshooting

Lifecycle States

- Created → Running → Paused → Restarted → Stopped → Removed
- docker stop sends SIGTERM (graceful) — docker kill sends SIGKILL (immediate)
- Exit codes reveal why a container stopped (0 = success, 137 = OOM killed)

Inspection Commands

- docker inspect — full JSON metadata (IP address, mounts, env vars)
- docker stats — live CPU, memory, and network usage per container

Cleanup

- docker system prune — removes stopped containers, dangling images, unused networks
- docker image prune -a — removes all images not used by any container

Copyright © 2026 Accenture. All rights reserved.

Inspecting and Debugging Containers

```
# Inspect container details (IP, mounts, environment)
docker inspect api --format '{{.NetworkSettings.IPAddress}}'

# Live resource usage
docker stats --no-stream
# CONTAINER   CPU %   MEM USAGE / LIMIT     MEM %
# api         0.15%   48.2MiB / 7.77GiB     0.61%

# Copy files between host and container
docker cp api:/app/logs ./local-logs

# View container's exit code
docker inspect api --format '{{.State.ExitCode}}'

# Clean up everything unused
docker system prune -f
```



Copyright © 2026 Accenture. All rights reserved.

How Docker Storage Works

OverlayFS Architecture

- Images are composed of read-only layers stacked on top of each other
- Each Dockerfile instruction (RUN, COPY) creates a new layer

Writable Container Layer

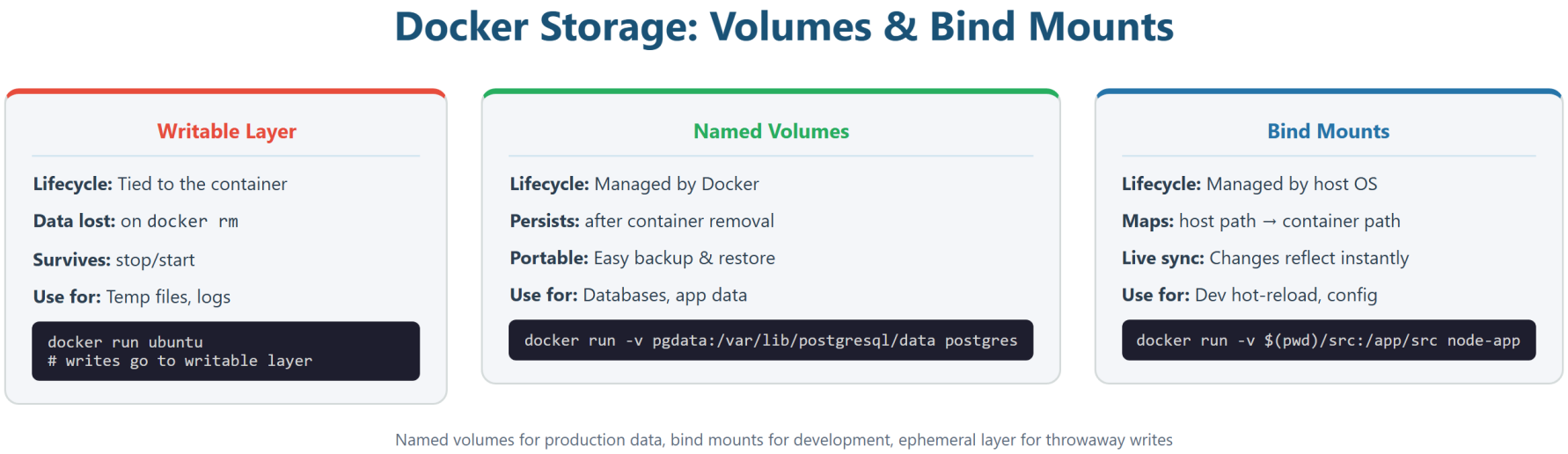
- When a container starts, a thin writable layer is added on top
- All file changes (creates, modifies, deletes) go to this writable layer
- This layer is ephemeral — lost when the container is removed

Production Rule

- Never store production data in the container writable layer — use volumes

Copyright © 2026 Accenture. All rights reserved.

Docker Storage Options



Copyright © 2026 Accenture. All rights reserved.

Working with Volumes

Named Volumes

- Managed by Docker — data persists even after container removal
- Best for databases and application data in production

Bind Mounts

- Map a host directory directly into the container filesystem
- Ideal for development — code changes reflect instantly without rebuild

Volume Commands

- docker volume create / ls / inspect / rm — manage named volumes
- docker run -v pgdata:/var/lib/postgresql/data postgres



Copyright © 2026 Accenture. All rights reserved.

Dockerfiles and Layer Caching

Section 4



Dockerfile Instructions

Build-Time Instructions

- FROM — base image (e.g., node:20-alpine, python:3.12-slim)
- WORKDIR — set working directory (created if not exists)
- COPY / ADD — copy files into image (prefer COPY unless ADD features are required)
- RUN — execute a command during build (e.g., npm ci, apt-get install)
- ARG — build-time variable (not available at runtime)

Runtime Instructions

- ENV — set environment variable (persists at runtime)
- EXPOSE — document which port the app listens on (informational only)
- USER — run the process as a non-root user for security
- HEALTHCHECK — define a health probe for orchestrators
- CMD vs ENTRYPOINT — default command vs fixed executable



Copyright © 2026 Accenture. All rights reserved.

Dockerfile for an Express App

```
FROM node:20-alpine
WORKDIR /home/node/app

# Copy dependency manifests first (cache-friendly)
COPY --chown=node:node package*.json ./
RUN npm ci --omit=dev

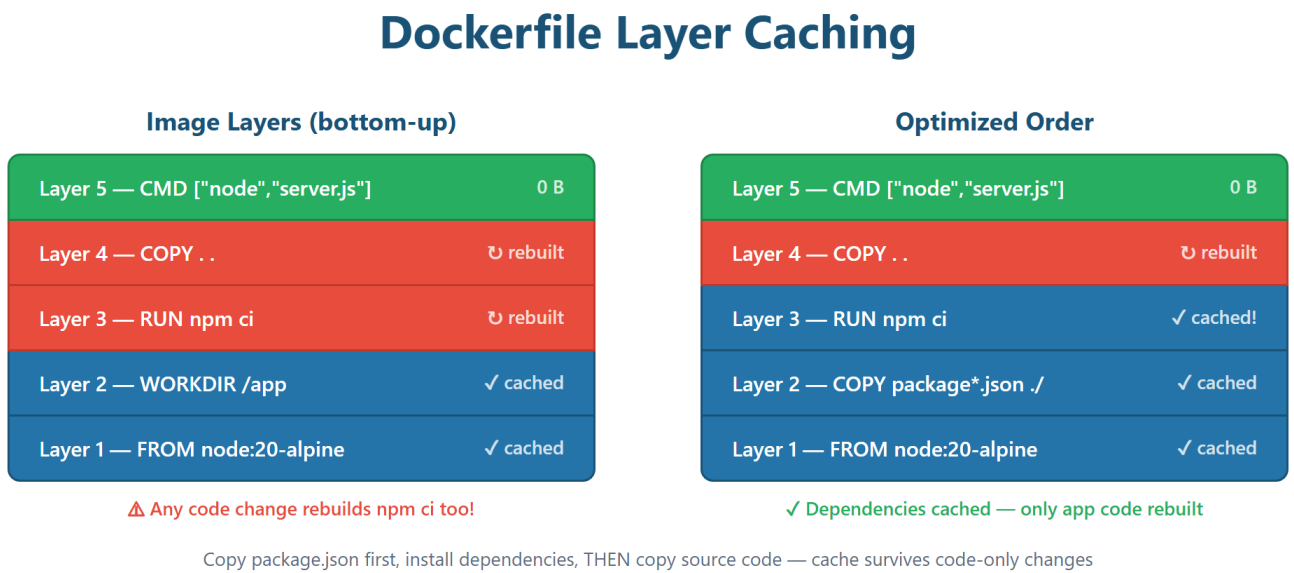
# Copy application source
COPY --chown=node:node . .

# Switch to non-root user for runtime security
USER node
ENV NODE_ENV=production
EXPOSE 3000
HEALTHCHECK CMD wget -q --spider http://localhost:3000/health || exit 1
CMD ["node", "server.js"]
```



Copyright © 2026 Accenture. All rights reserved.

Dockerfile Layer Caching



Copyright © 2026 Accenture. All rights reserved.

Cache-Efficient Dockerfiles

How Caching Works

- Each instruction creates a layer — Docker caches layers whose inputs haven't changed
- When a layer changes, all subsequent layers must be rebuilt

Optimization Strategy

- Put rarely-changing instructions first (FROM, WORKDIR, system packages)
- Copy package.json / requirements.txt before source code
- Install dependencies in a separate RUN step — cached until manifests change
- Copy application source last — the layer that changes most often



Copyright © 2026 Accenture. All rights reserved.

.dockerignore and Build Context

```
# .dockerignore — keep the build context small and secure
node_modules
.git
.env
*.log
coverage/
dist/
Dockerfile
docker-compose*.yml
README.md

# Build the image (. = build context = current directory)
docker build -t my-express-app:1.0.0 .

# .dockerignore prevents secrets and large dirs from
# being sent to the Docker daemon as build context
```



Copyright © 2026 Accenture. All rights reserved.

Base Image Selection

Alpine Variants (node:20-alpine)

- ~50 MB vs ~350 MB for full images — smaller attack surface
- Uses musl libc — some native modules may need rebuilding

Slim Variants (node:20-slim)

- ~80 MB — removes docs and non-essential tools from Debian
- Better compatibility with native modules — use when native dependencies are required

Distroless (gcr.io/distroless/nodejs20)

- No shell, no package manager — smallest attack surface possible
- Harder to debug in production — no shell access for troubleshooting
- Pin image tags (node:20.11.1-alpine) — never rely on :latest in production

Copyright © 2026 Accenture. All rights reserved.

Multi-Stage Builds

Section 5



Why Multi-Stage Builds?

The Problem with Single-Stage

- Build tools, compilers, and devDependencies bloat the final image
- A typical Node.js build image can be 1+ GB with TypeScript, webpack, etc.

The Multi-Stage Solution

- Stage 1 (builder): install everything, compile, run tests
- Stage 2 (runtime): start from a clean base, copy only production artifacts
- Build tools and source code are discarded — never shipped to production

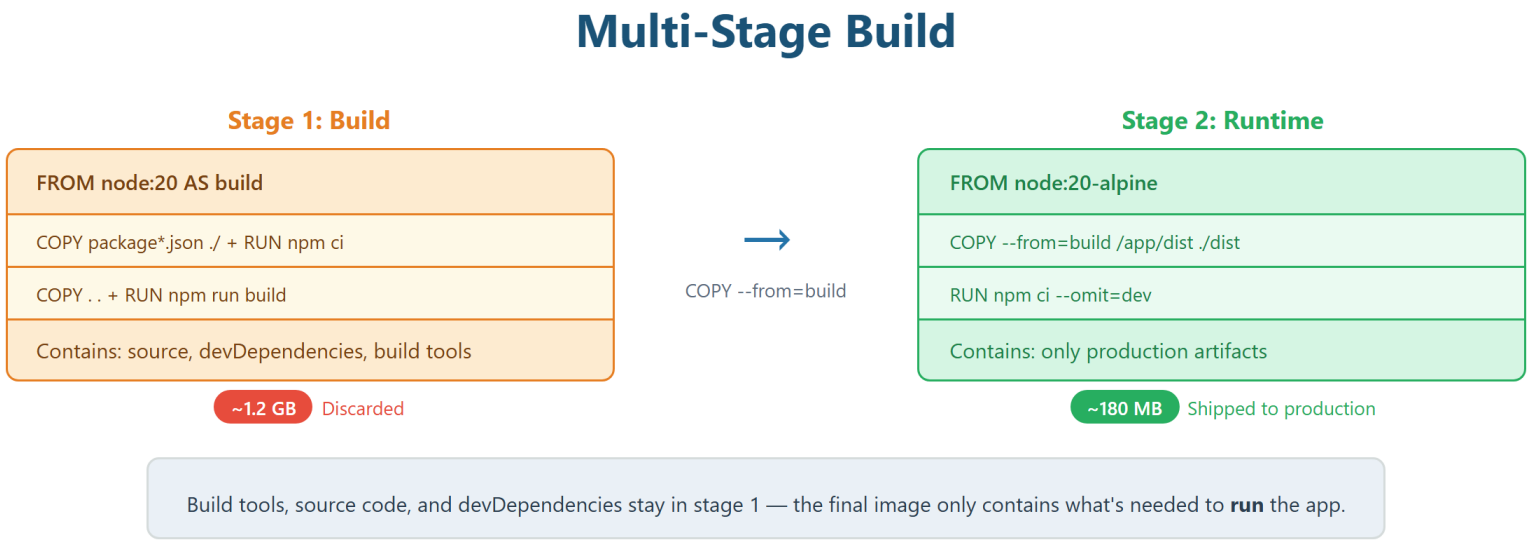
Benefits

- Dramatically smaller images, faster pulls, reduced attack surface



Copyright © 2026 Accenture. All rights reserved.

Multi-Stage Build Flow



Copyright © 2026 Accenture. All rights reserved.

Multi-Stage Dockerfile

```
# Stage 1: Build
FROM node:20 AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
RUN npm test

# Stage 2: Production runtime
FROM node:20-alpine
USER node
WORKDIR /home/node/app
COPY --from=build --chown=node:node /app/dist ./dist
COPY --from=build --chown=node:node /app/package*.json ./
RUN npm ci --omit=dev
ENV NODE_ENV=production
EXPOSE 3000
CMD ["node", "dist/server.js"]
```



Copyright © 2026 Accenture. All rights reserved.

Build and Compare Image Sizes

```
# Build the multi-stage image
docker build -t my-app:multi .

# Compare image sizes
docker images my-app
# REPOSITORY   TAG      SIZE
# my-app       single   1.24GB
# my-app       multi    178MB

# Tag with semantic version and push
docker tag my-app:multi registry.example.com/my-app:1.2.0
docker push registry.example.com/my-app:1.2.0

# Never use :latest in production — pin exact versions
# docker pull registry.example.com/my-app:1.2.0
```



Copyright © 2026 Accenture. All rights reserved.

Docker Compose

Section 6



Docker Compose Concepts

Multi-Container Applications

- Real apps need multiple services — API, database, cache, reverse proxy
- Compose defines all services in a single compose.yaml file
- Compose is a development tool — not a production orchestrator

Networking

- Compose auto-creates a network — services resolve each other by name
- Only ports listed in 'ports:' are exposed to the host — internal comms use container ports

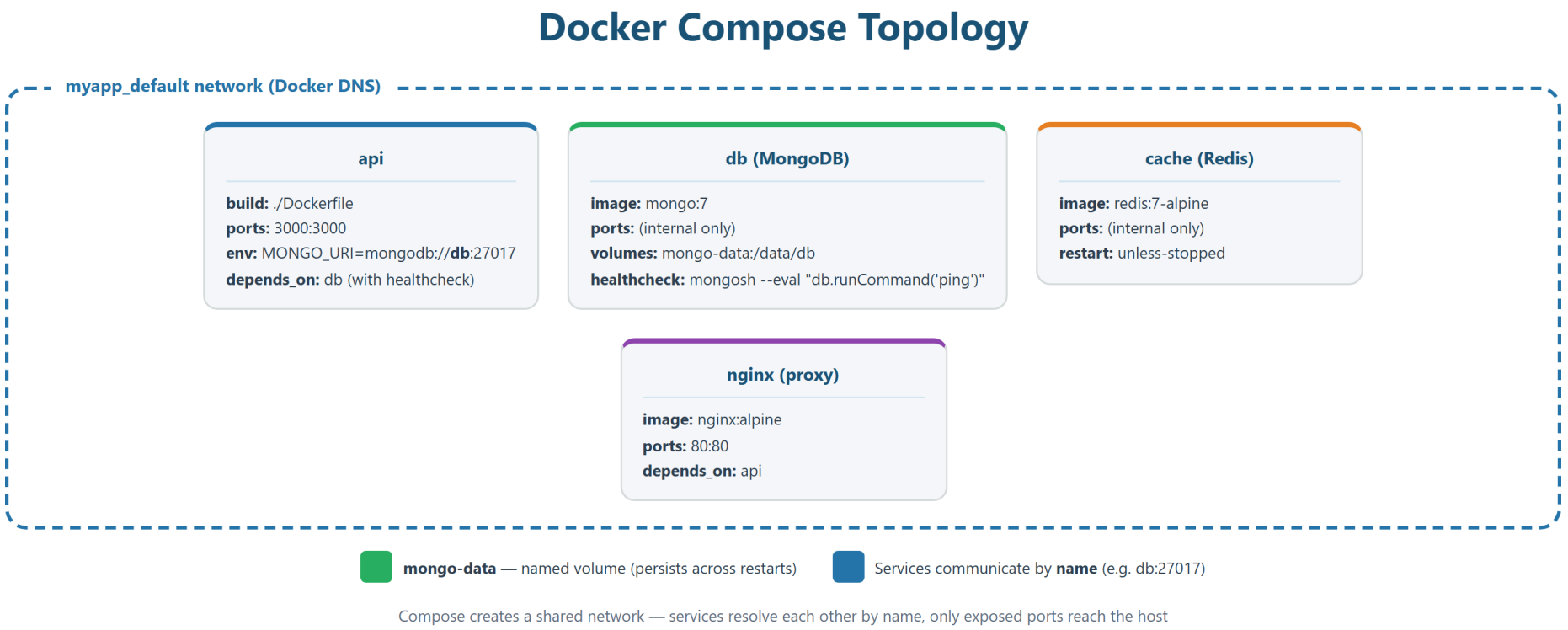
Startup Order and Readiness

- depends_on controls startup order but does NOT wait for application readiness
- Use depends_on with condition: service_healthy and a HEALTHCHECK for true readiness



Copyright © 2026 Accenture. All rights reserved.

Docker Compose Topology



Copyright © 2026 Accenture. All rights reserved.

compose.yaml (Compose V2)

```yaml
# No 'version:' field needed — Compose V2 uses the Compose Specification
services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      MONGODB_URI: mongodb://db:27017/myapp
    depends_on:
      db:
        condition: service_healthy
  db:
    image: mongo:7
    volumes:
      - mongo-data:/data/db
    healthcheck:
      test: mongosh --eval "db.runCommand('ping').ok" --quiet
      interval: 10s
      retries: 3
volumes:
  mongo-data:
```



Copyright © 2026 Accenture. All rights reserved.

Environment Variables and Secrets

Environment Configuration

- Use env_file to load variables from a .env file (keep secrets out of compose.yaml)
- Prefer mapping syntax (KEY: value) over array syntax (- KEY=value)

Secrets Handling

- Never bake credentials into images — use env vars or Docker secrets
- Add .env to .gitignore and .dockerignore — prevent accidental exposure

Restart Policies

- restart: unless-stopped — auto-restart on crash, stop only when explicitly asked



Copyright © 2026 Accenture. All rights reserved.

Compose Commands

```
# Start all services in detached mode
docker compose up -d

# View running services and their health status
docker compose ps

# Follow logs for a specific service
docker compose logs -f api

# Rebuild after code changes
docker compose up -d --build

# Stop and remove containers, networks
docker compose down

# Stop and also remove named volumes (CAUTION: deletes data)
docker compose down -v
```



Copyright © 2026 Accenture. All rights reserved.

Development with Compose

```
# compose.override.yaml — auto-merged for local dev
services:
  api:
    build:
      context: .
      target: build    # use build stage for dev tools
    volumes:
      - ./src:/home/node/app/src    # bind mount for hot-reload
    environment:
      NODE_ENV: development
    command: npx nodemon src/server.js

# Start dev environment (merges compose.yaml + override)
docker compose up

# Production: explicit file (ignores override)
docker compose -f compose.yaml up -d
```



Copyright © 2026 Accenture. All rights reserved.

Security and Production

Section 7

Container Security Best Practices

Run as Non-Root

- USER node (or create a dedicated user) — prevents container breakout escalation
- Most official images include a non-root user (node, postgres, nginx)

Minimize Attack Surface

- Use small base images (alpine, slim, distroless) — fewer packages = fewer CVEs
- Drop capabilities: --cap-drop=ALL, then add back only what's needed
- Use read-only filesystem where possible: --read-only flag

No Secrets in Images

- Never COPY .env or hardcode credentials in Dockerfiles
- Use environment variables, Docker secrets, or a vault at runtime



Copyright © 2026 Accenture. All rights reserved.

Security Scanning

```
# Scan an image for known vulnerabilities
docker scout cves my-app:1.2.0

# Quick vulnerability summary
docker scout quickview my-app:1.2.0
# 0 critical, 2 high, 5 medium, 12 low

# Compare two image versions for new vulnerabilities
docker scout compare my-app:1.2.0 --to my-app:1.1.0

# Trivy — widely used open-source scanner in production
trivy image my-app:1.2.0

# Pin image by digest for reproducible builds
FROM node:20-alpine@sha256:abc123def456...
```



Copyright © 2026 Accenture. All rights reserved.

Image Tagging Strategy

Semantic Versioning

- Tag every release with a version (v1.2.0) — enables precise rollbacks
- Also tag with git SHA for traceability (my-app:abc123f)

Avoid :latest in Production

- :latest is mutable — pulling it at different times gives different images
- Pin exact versions in Dockerfiles and deployment manifests

Registry Workflow

- docker login → docker tag → docker push to your registry
- CI/CD pipelines should build, scan, tag, and push automatically

Copyright © 2026 Accenture. All rights reserved.

Production Registry Workflow

```
# Authenticate to a private registry
docker login registry.example.com

# Tag with semantic version and git SHA
docker tag my-app:latest registry.example.com/my-app:1.2.0
docker tag my-app:latest registry.example.com/my-app:$(git rev-parse --short HEAD)

# Push both tags
docker push registry.example.com/my-app:1.2.0
docker push registry.example.com/my-app:abc123f

# Pull a specific version on any server
docker pull registry.example.com/my-app:1.2.0
docker run -d -p 3000:3000 registry.example.com/my-app:1.2.0
```



Copyright © 2026 Accenture. All rights reserved.

Production Considerations

Resource Limits

- Set memory and CPU limits — prevents a single container from starving the host
- Enforced by cgroups at the kernel level
- docker run --memory=512m --cpus=1.5 my-app

Logging and Monitoring

- Write logs to stdout/stderr — let Docker's logging driver handle collection
- Use docker stats or integrate with Prometheus/Grafana for monitoring

Beyond Compose

- Compose is for development — use an orchestrator in production
- Kubernetes is the industry standard for production container orchestration

Copyright © 2026 Accenture. All rights reserved.

Key Takeaways

Key Takeaways

1. Containers are Linux-native process isolation — namespaces, cgroups, OverlayFS
2. Docker is tooling, not the only solution — containerd, Podman, CRI-O also run containers
3. Containers share the host kernel — lightweight and fast compared to VMs
4. Images are layered filesystems — each Dockerfile instruction adds a read-only layer
5. Multi-stage builds are required for production — separate build tools from runtime
6. Use volumes for persistent data — never rely on the container writable layer
7. Compose is for development — not a production orchestrator
8. Use Kubernetes for production container orchestration
9. Follow strict security practices — non-root, cap-drop, scan for CVEs, no secrets in images



Copyright © 2026 Accenture. All rights reserved.